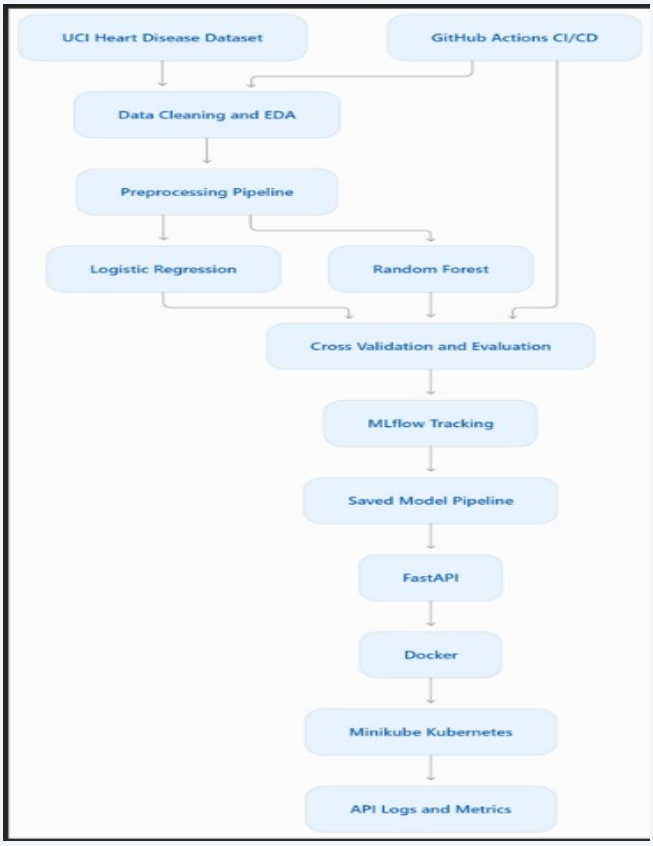
HEART DISEASE CLASSIFICATION

End-to-End MLOps Project Report

Data preparation • Model development • Experiment tracking • API • CI/CD • Containerization • Kubernetes

Report detail	Value
Project	heart-disease-mlops
Programming environment	Python 3.11 / Anaconda (Windows development environment)
Dataset source	UCI Machine Learning Repository — Heart Disease (Cleveland subset used by project code)
Experiment tracking	MLflow with SQLite backend (local development)
API framework	FastAPI / Uvicorn
Deployment target	Docker and Minikube Kubernetes (local demonstration)
Prepared for	Chaitanya Patil (2025af05025)

PROJECT ARCHITECTURE OVERVIEW



1.1. Executive Summary

Project purpose, scope, and intended outcomes

This project demonstrates a practical, end-to-end Machine Learning Operations (MLOps) workflow for a binary classification task using the UCI Heart Disease dataset. The implementation brings together data preparation, exploratory analysis, model comparison, experiment tracking, automated checks, API serving, containerization, and local Kubernetes deployment.

Objectives

Create a repeatable data preparation step that obtains the dataset, handles missing-value markers, and constructs a binary target according to the project code.

Explore feature distributions, target balance, and correlations to understand the data before training.

Train and compare Logistic Regression and Random Forest models using a reproducible train/test split and cross-validation as configured.

Record experiment metrics and model artifacts with MLflow using a SQLite-backed tracking store for local development.

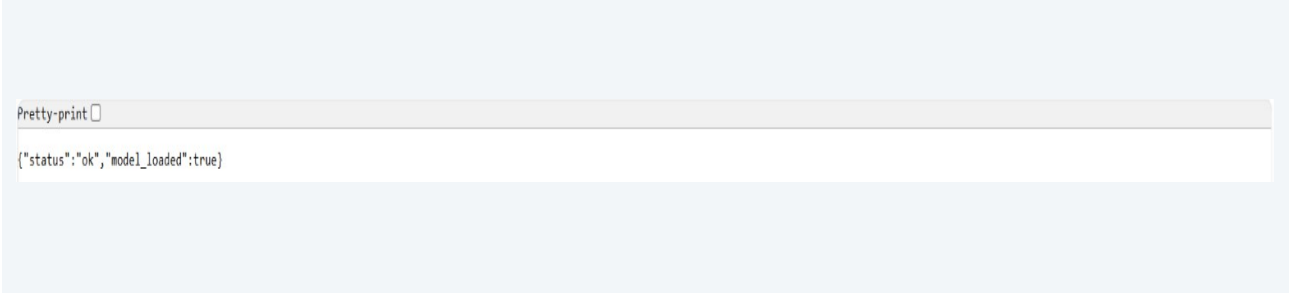
Expose model inference through a FastAPI service with health and metrics endpoints.

Apply code-quality checks and unit tests, then package and run the service with Docker and Kubernetes.

Scope and deliverables

The intended deliverables are source code, dependency definitions, data download/processing instructions, tests, a GitHub Actions workflow, a Dockerfile, Kubernetes manifests, screenshots, and this written report. The workflow described here is continuous integration: it lints, trains, tests, and uploads available artifacts. Automated image publishing or deployment is a separate enhancement unless added to the pipeline.

Current evidence to insert



No model performance values are asserted in this report because the final training output has not been supplied here. Populate the results section with values copied from your actual MLflow run and training output.

1.2. Dataset and Data Preparation

Source, target definition, and repeatability

Dataset source

The project uses the Heart Disease dataset from the UCI Machine Learning Repository. The data preparation module (src/data.py) is designed to retrieve the dataset using the ucimlrepo package, normalize missing-value markers, convert columns to numeric where appropriate, remove rows without a usable target, and save a processed local CSV.

Target construction

The project code maps the original target to a binary label by treating target values greater than zero as the positive class and zero as the negative class. This conversion is part of the project's implementation and should be documented consistently in the code, report, and API explanation.

Data preparation sequence

2. Retrieve the dataset from its stated source.
3. Replace string missing markers such as '?' with missing values.
4. Convert feature columns to numeric values where appropriate.
5. Remove rows where the target is missing.
6. Create the binary target label using the configured mapping.
7. Write the processed dataset to data/heart.csv for subsequent EDA and training.

Reproducibility and data handling

The data download script should be committed to GitHub, while generated data may be excluded from version control by .gitignore. This keeps the repository lighter and allows CI to regenerate the dataset. The README should include the exact download command and note that internet access is required for a fresh download.

Insert screenshot of the data preparation command completing, or a notebook/table showing rows, columns, data types, and missing values.

```
(heart-mlops) C:\Users\Prakash\heart-disease-mlops>python -c "import pandas as pd; d=pd.read_csv('data/heart.csv'); print(d.head()); print(d.shape); print(d
.isna().sum()); print(d['target'].value_counts())"
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope   ca  thal  target
0   63    1   1       145   233    1        2      150      0      2.3      3  0.0   6.0       0
1   67    1   4       160   286    0        2      108      1      1.5      2  3.0   3.0       1
2   67    1   4       120   229    0        2      129      1      2.6      2  2.0   7.0       1
3   37    1   3       130   250    0        0      187      0      3.5      3  0.0   3.0       0
4   41    0   2       130   204    0        2      172      0      1.4      1  0.0   3.0       0
(303, 14)
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
target      0
dtype: int64
target
0    164
1    139
Name: count, dtype: int64
```

Validation notes: record the final row count, feature count, target class counts, and any rows removed after running the actual pipeline.

8.3. Exploratory Data Analysis

Understanding distributions and relationships

Exploratory Data Analysis (EDA) is used to inspect the data before fitting models. The project's EDA script is intended to generate a target class-balance summary, feature distribution plots, and a correlation heatmap. These visualizations help identify skewed variables, possible data-quality issues, class imbalance, and relationships among numeric features.

Recommended interpretation

Class balance: compare the number of positive and negative examples; imbalance can affect the interpretation of accuracy.

Feature distributions: inspect ranges, unusual values, and skew. Do not remove observations solely because they appear unusual without a defensible reason.

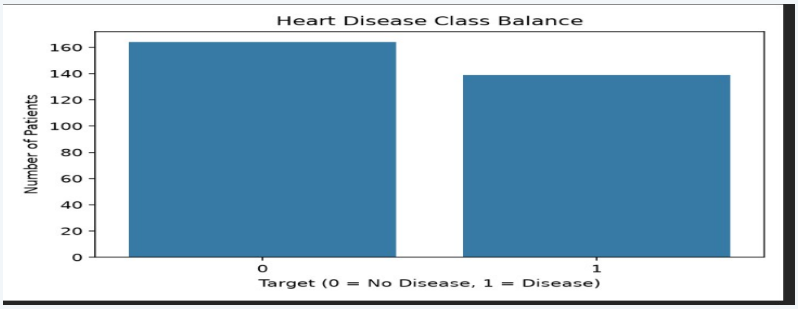
Correlation heatmap: use it as an exploratory aid, not proof of causality. Correlation among predictors can influence some model types and coefficient interpretation.

Data leakage: perform preprocessing inside the scikit-learn pipeline so statistics such as imputation and scaling are learned from training folds rather than the full dataset.

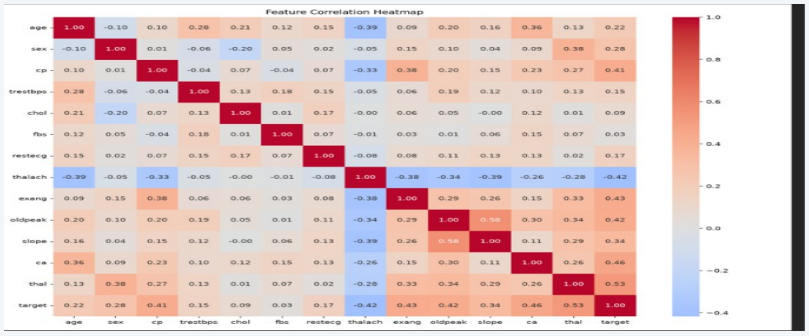
EDA outputs

Output	Purpose	Evidence to record
Target class balance	Understand label distribution	Counts and/or bar chart
Feature histograms	Inspect shape and range of features	Representative plots
Correlation heatmap	Review linear relationships among numeric columns	Heatmap image
Data-quality summary	Identify missingness and conversion issues	Rows/columns and missing counts

[EDA FIGURE 1 — TARGET CLASS BALANCE]



[CORRELATION HEATMAP]



9.4. Model Development and Evaluation

Baseline comparison and validation

The training module is designed to compare Logistic Regression and Random Forest classifiers. Logistic Regression provides a relatively interpretable linear baseline, while Random Forest can represent nonlinear relationships and interactions. Their performance should be compared using the same split strategy and metrics.

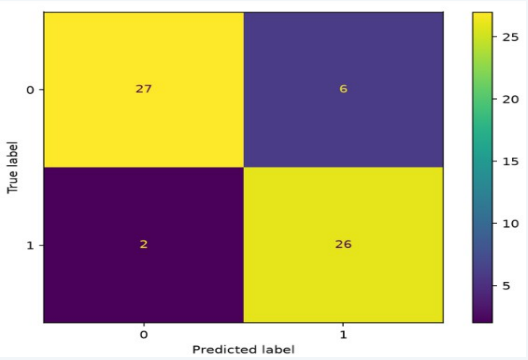
Pipeline design

The project uses scikit-learn pipelines and preprocessing components so that imputation, scaling, and categorical-feature handling can be applied consistently. Pipeline-based preprocessing also reduces the risk of leakage during cross-validation. The feature list and categorical columns must remain consistent between training and the FastAPI request schema.

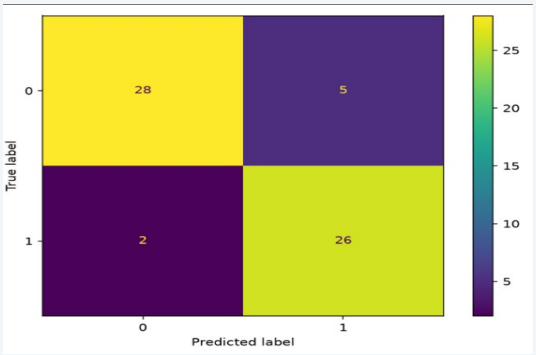
Evaluation measures

Metric	What it measures	Interpretation caution
Accuracy	Fraction of predictions that are correct	Can hide poor minority-class performance
Precision	Fraction of predicted positives that are true positives	False positives affect this value
Recall	Fraction of actual positives detected	False negatives affect this value
ROC-AUC	Ranking quality across classification thresholds	Requires appropriate probability/score output

[TRAINING OUTPUT / MODEL COMPARISON]



Logistic Regression



Random Forest

```
Results for Logistic Regression
{
  "cv_accuracy": 0.8551870748299321,
  "cv_precision": 0.8730744935655421,
  "cv_recall": 0.8098814229249012,
  "cv_roc_auc": 0.9024779568257829,
  "test_accuracy": 0.8688524590163934,
  "test_precision": 0.8125,
  "test_recall": 0.9285714285714286,
  "test_roc_auc": 0.9664502164502166
}
MLflow run ID: c47c5ff16d97428ba36de8110cb4f317
2026/10/20 22:30:16 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats
 rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' f
ormat. For more information, see: https://scikit-learn.org/stable/model_persistence.html

Results for Random Forest
{
  "cv_accuracy": 0.8180272108843537,
  "cv_precision": 0.818312080076786,
  "cv_recall": 0.782608695652174,
  "cv_roc_auc": 0.9017122169296081,
  "test_accuracy": 0.8852459016393442,
  "test_precision": 0.8387096774193549,
  "test_recall": 0.9285714285714286,
  "test_roc_auc": 0.9512987012987013
}
MLflow run ID: df8e7307105f45c2a83f541d5bc84cd1
```

Model selection should be justified using the evaluation evidence and the intended error trade-offs. Do not describe a model as clinically accurate based only on performance on this small public dataset.

10. 5. MLflow Experiment Tracking

Logging parameters, metrics, and artifacts

MLflow is used to organize and compare model-training runs. In the local Windows environment, the tracking URI is configured as sqlite:///mlflow.db. The training script should log the experiment name, model/run details, evaluation metrics, and available plots or model artifacts. The MLflow UI is launched separately so that experiments can be inspected in a browser.

Local workflow

11. Activate the heart-mlops Conda environment.
12. Run data preparation and training from the project root.
13. Start the UI with: mlflow ui --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5000
14. Open http://127.0.0.1:5000 and select the relevant experiment.
15. Compare run metrics and inspect logged artifacts before deciding which model to use.

Tracking benefits

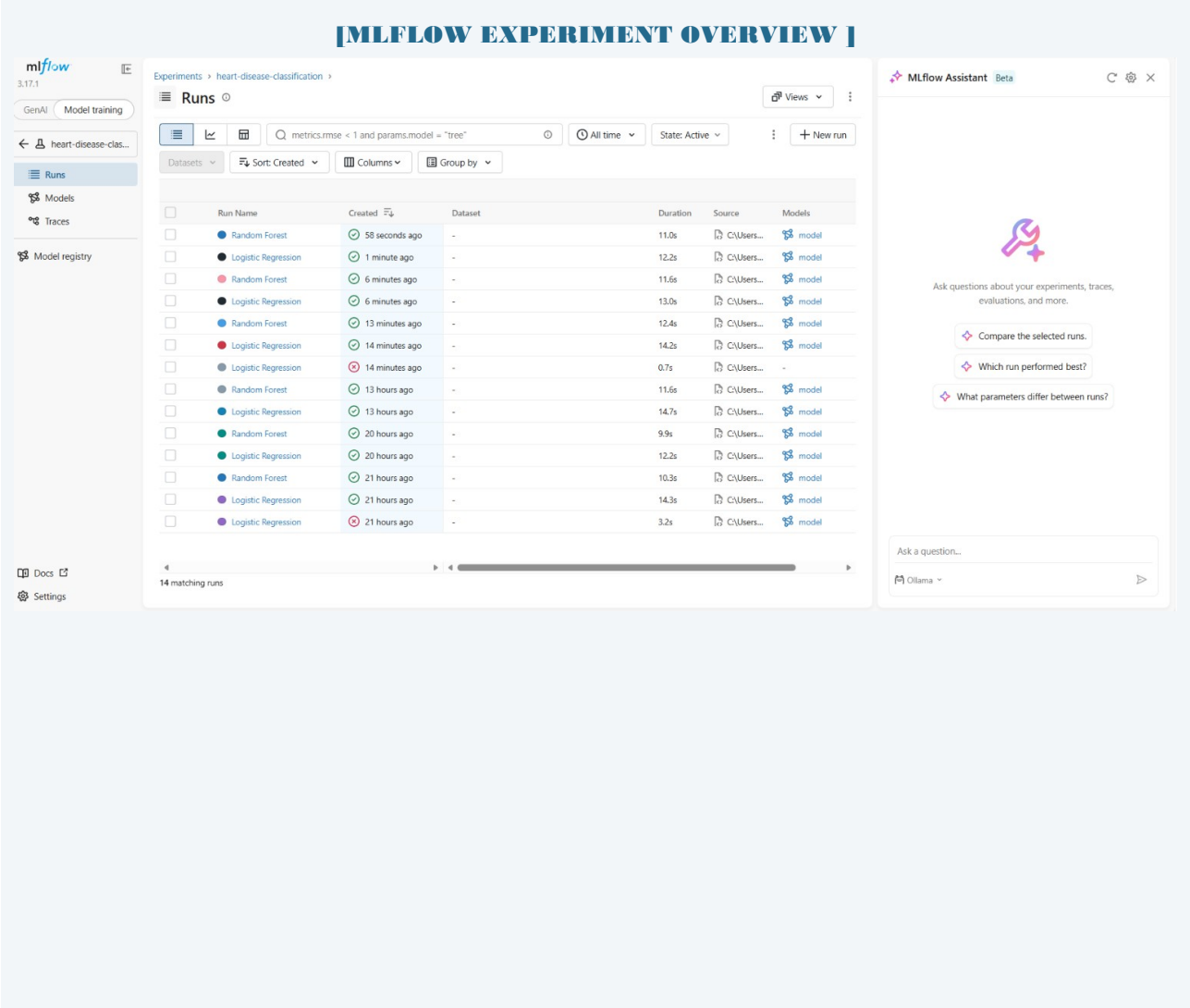
Creates a record of model runs and their evaluation results.

Supports comparison across model types and parameter settings.

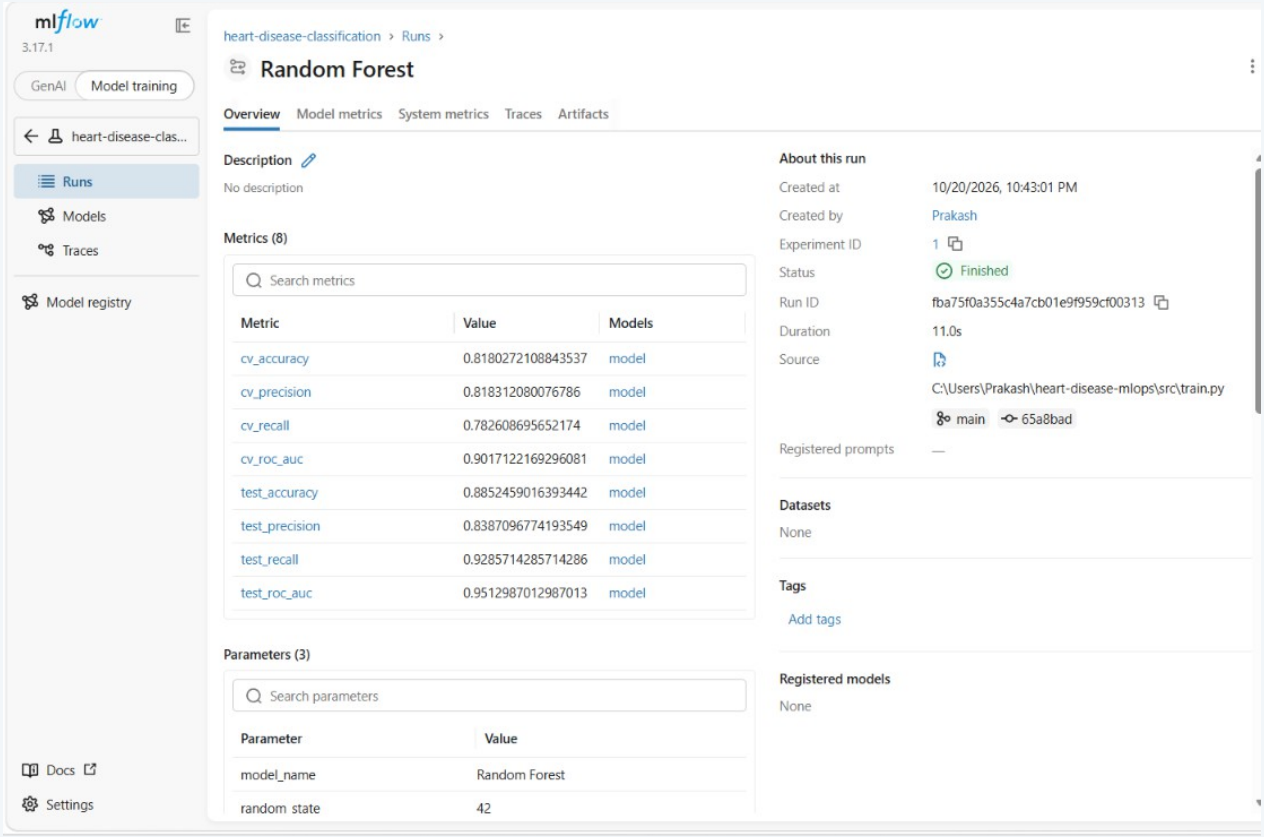
Helps reviewers reproduce the reported result when the code, dependencies, data source, and run configuration are preserved.

Separates model experimentation from API serving, although this prototype still uses local storage rather than a centralized production tracking service.

[MLFLOW EXPERIMENT OVERVIEW]



[MLFLOW RUN DETAILS]



If the UI displays no runs, verify that training and the UI use the same tracking URI and working directory.

16. 6. API, Inference, and Monitoring

Serving predictions with FastAPI

The API module (src/api.py) loads the serialized model and exposes HTTP endpoints through FastAPI. The /health endpoint indicates whether the service is healthy and whether the model was loaded. The /predict endpoint accepts a JSON record containing the 13 configured Cleveland dataset features. A /metrics endpoint exposes Prometheus-compatible request metrics where configured.

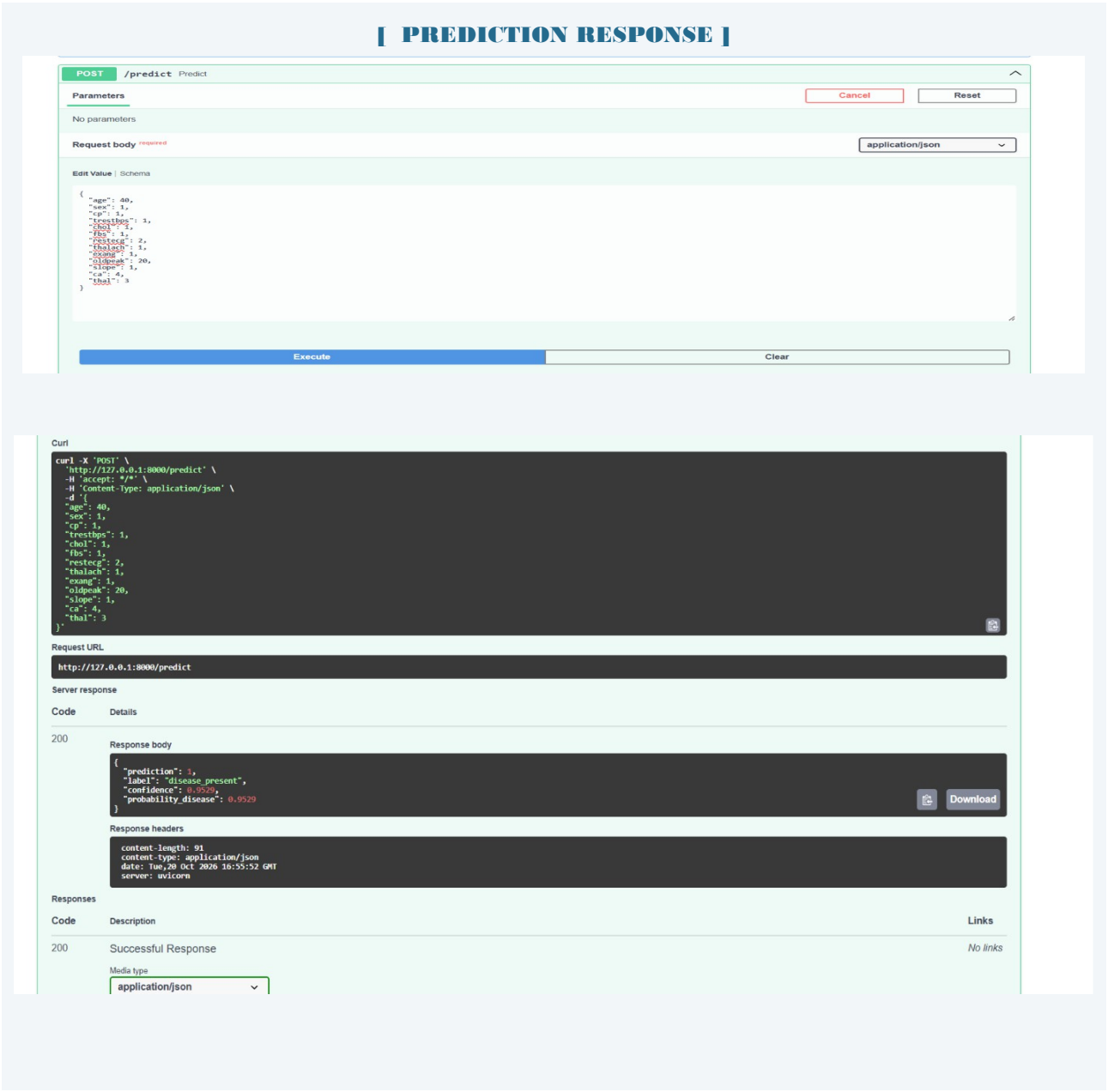
API test workflow

17. Start the service from the repository root with: uvicorn src.api:app --reload
18. Open http://127.0.0.1:8000/docs.
19. Check GET /health and confirm model_loaded is true.
20. Use POST /predict with a valid sample request matching the Pydantic schema.
21. Inspect GET /metrics and the server logs to verify basic observability.

Example request fields

The schema includes age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, and thal. Values should match the expected feature meanings and coding used by the source dataset. Request validation can reject malformed inputs, but schema validation alone does not establish clinical appropriateness.





Monitoring in this prototype is intentionally lightweight. Prometheus metrics and container logs provide basic evidence of activity, but production monitoring would also require alerting, access controls, retention policies, error analysis, and privacy protections.

22. 7. Testing, Code Quality, and CI/CD

Automated verification through GitHub Actions

The project includes pytest tests for data handling and API behavior, and Ruff is used to check source-code quality and import organization. The GitHub Actions workflow (.github/workflows/ci.yml) is configured to run on pushes and pull requests targeting main. It checks out the repository, installs dependencies, lints the code, downloads/prepares the data, trains models, runs tests, and uploads available artifacts.

Local verification commands

```
ruff check src tests
python -m src.data
python -m src.train
python -m pytest -v
```

CI behavior and limitations

A green workflow run means the configured checks passed for that commit; it does not prove the model is clinically safe.

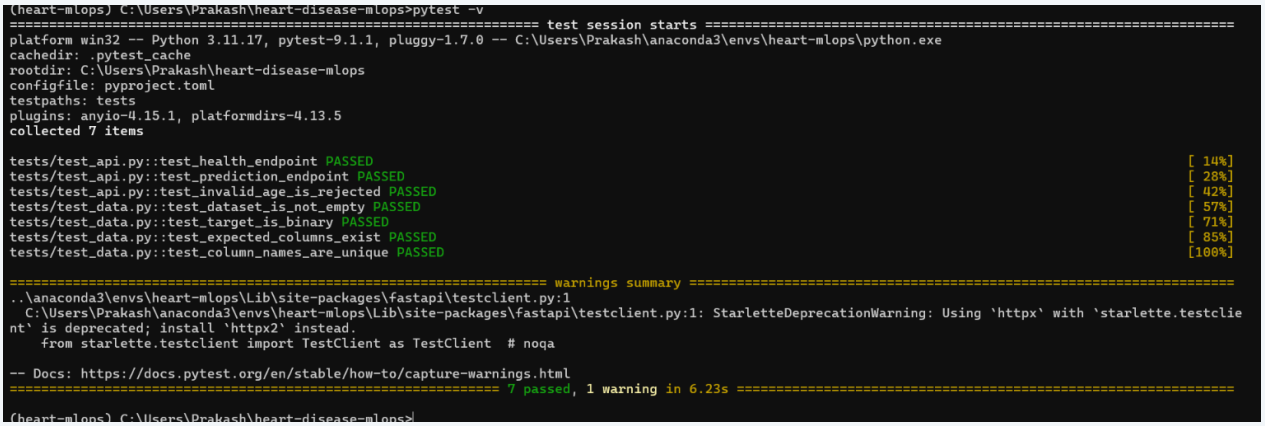
The workflow must use the same dependency versions and commands as the repository documentation.

Dataset download requires network access in the hosted runner.

SQLite and MLflow artifact paths should be configured deliberately if artifacts are expected to be uploaded.

The current CI workflow does not automatically publish a Docker image or deploy to Kubernetes unless those stages are added.

[GITHUB ACTIONS SUCCESSFUL RUN]



23. 8. Docker and Kubernetes Deployment

Container packaging and local cluster validation

Docker packages the API code, runtime dependencies, and saved model into a container image. Kubernetes deployment is demonstrated locally with Minikube. The Kubernetes manifest defines the application deployment and service and can include health probes. On Windows with the Docker driver, the terminal used by `minikube service --url` may need to remain open while the service tunnel is active.

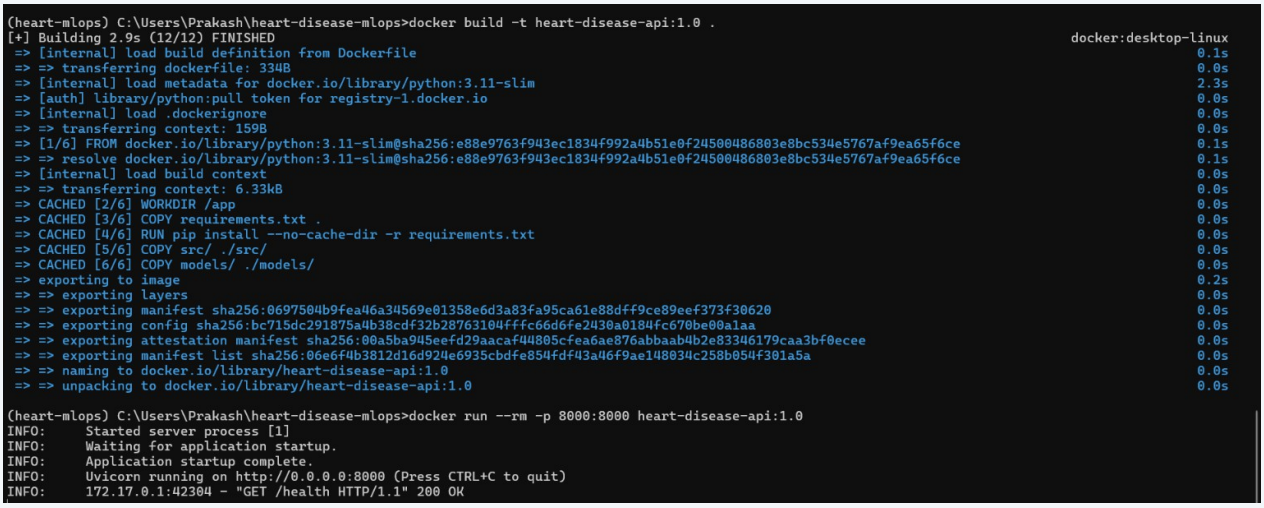
Docker workflow

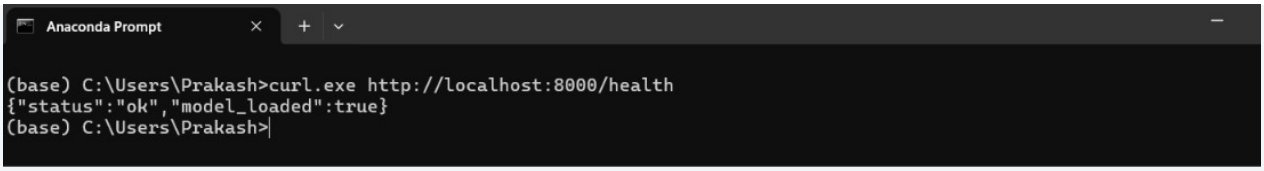
24. Confirm that Docker Desktop is running.
25. Ensure Dockerfile, requirements.txt, src/, and models/model.joblib are present.
26. Build the image: docker build -t heart-disease-api:1.0 .
27. Run locally: docker run --rm -p 8000:8000 heart-disease-api:1.0
28. Verify /health and /docs through localhost.

Minikube workflow

29. Start the cluster: minikube start --driver=docker.
30. Confirm the node is Ready: kubectl get nodes.
31. Build the image for Minikube: minikube image build -t heart-disease-api:1.0 .
32. Apply the manifest: kubectl apply -f deployment/deployment.yaml.
33. Inspect resources: kubectl get pods and kubectl get services.
34. Expose the API with `minikube service heart-disease-api --url` or use `kubectl port-forward`.

[DOCKER BUILD / CONTAINER HEALTH]





The Minikube URL is a local access URL, not a public production endpoint. For a persistent public deployment, hosting, network exposure, TLS, authentication, and operational monitoring would need to be configured separately.

35. 9. Conclusion, Limitations, and Submission Checklist

Final review and evidence

Conclusion

This project demonstrates the main stages of an applied MLOps workflow: data preparation, exploratory analysis, model comparison, experiment tracking, API serving, automated tests, CI, containerization, and local Kubernetes deployment. It provides a foundation for learning how model code can be moved from an experiment into a reproducible service. The final quality of the submission depends on running the complete pipeline successfully and replacing placeholders with genuine evidence from the implementation.

Limitations and responsible use

The dataset is small and may not represent current clinical practice or all populations.

The model is a learning exercise, not a validated clinical tool or medical device.

Metrics on a held-out subset do not demonstrate generalization to hospitals, geographies, or real-world populations.

The local development setup is not production-hardened; secrets, access control, TLS, privacy, audit, drift monitoring, and incident response need further design.

Report actual model metrics, test results, and deployment outcomes; do not invent or estimate missing results.

Final submission checklist

Item	Before submission
Source code, requirements, README	Verify files are committed and pushed
Data download/processing	Run successfully and document source
EDA and model evaluation	Include actual figures and metric values
MLflow	Include screenshots of experiment and run details
Tests and lint	Capture passing output and successful CI run
Docker and Kubernetes	Capture successful build and running resources
Screenshots	Replace all placeholders with genuine screenshots
Report	Review pagination, add name/course/date, and export PDF if required
GitHub repository	Push latest report and evidence; verify Actions status

```
(base) C:\Users\Prakash>kubectl describe deployment heart-disease-api
Name:                   heart-disease-api
Namespace:              default
CreationTimestamp:      Tue, 20 Oct 2026 14:47:18 +0530
Labels:                 <none>
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=heart-disease-api
Replicas:               1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=heart-disease-api
  Containers:
   api:
    Image:      heart-disease-api:1.0
    Port:       8000/TCP
    Host Port:  0/TCP
    Limits:
      cpu:     500m
      memory:  512Mi
    Requests:
      cpu:        100m
      memory:     128Mi
    Liveness:     http-get http://:8000/health delay=10s timeout=1s period=20s #success=1 #failure=3
    Readiness:    http-get http://:8000/health delay=5s timeout=1s period=10s #success=1 #failure=3
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   heart-disease-api-7f6985775b (1/1 replicas created)
Events:          <none>

(base) C:\Users\Prakash>kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
heart-disease-api-7f6985775b-qz7qb   1/1     Running   0          8h    10.244.0.3   minikube   <none>           <none>

(base) C:\Users\Prakash>
```

References

UCI Machine Learning Repository, Heart Disease dataset: https://archive.ics.uci.edu/dataset/45/heart+disease

Project implementation and outputs: local source code, MLflow run records, GitHub Actions logs, and Kubernetes resources. Add repository URL here: [paste GitHub repository URL].